

Маршрут навчальної практики з об'єктно-орієнтованого програмування

Цей документ містить орієнтовний план роботи під час навчальної практики з ООП. Він призначений для того, щоб ви могли крок за кроком побудувати свій застосунок від UML-моделі до протестованого та задокументованого продукту.

[!TIP] **Пам'ятайте:** Головний критерій успішності - це реалізований, архітектурно продуманий і покритий тестами мініпроект. Ви можете адаптувати цей план під обрану предметну область (RPG, IoT, банк, LMS тощо), зберігаючи загальну логіку розділів та ключові принципи ООП.

ТЕМИ ДО ІНТЕГРОВАНИХ ПРОЄКТІВ (ВАРІАНТИ ДЛЯ СТУДЕНТІВ)

Для проходження практики студенту необхідно обрати один із варіантів предметної області та розвивати його протягом усіх занять:

1. **Консольний RPG-симулятор** (сутності: персонажі, монстри, зброя; патерни: Factory для ворогів, Strategy для унікальних атак, Observer для логування бою).
2. **Система «Розумний будинок» (IoT Simulator)** (сутності: сенсори, контролери, прилади; патерни: Observer для датчиків, Facade для панелі управління, Decorator для налаштувань).
3. **Трекер завдань (Канбан-дошка)** (сутності: завдання, епіки, користувачі; патерни: State для статусів задач, Composite для дерева підзадач, Factory для типів тасок).
4. **Емулятор банківської системи** (сутності: рахунки, транзакції, депозити; патерни: Strategy для нарахування відсотків, Command для операцій/відміни, Observer для сповіщень).
5. **Система кафе / ресторану** (сутності: замовлення, меню, персонал; патерни: State для обробки замовлення, Builder для комплексних обідів, Singleton для системи черг).
6. **Система логістики та автопарку** (сутності: транспорт, маршрути, вантажі; патерни: Strategy для розрахунку вартості, Factory Method для типів авто, Decorator для страховки).
7. **Управління готелем (Бронювання)** (сутності: номери, клієнти, послуги; патерни: Decorator для додаткових послуг номера, Builder для формування фінального рахунку/чека).
8. **Медична інформаційна система** (сутності: лікарі, пацієнти, медичні картки; патерни: Singleton для реєстру клініки, Observer для відслідковування черги, Facade для запису).
9. **Консольна покрокова стратегія** (сутності: юніти, міста, ресурси; патерни: Factory для юнітів, Strategy для алгоритмів AI противника, Composite для формування армій).
10. **Електронна бібліотека з підписками** (сутності: книги, читачі, абонементи; патерни: Decorator для преміум-підписок, Strategy для систем штрафних санкцій).
11. **Музичний плеєр та редактор плейлистів** (сутності: треки, альбоми, плейлисти, виконавці; патерни: Iterator для обходу плейлиста, Strategy для режимів відтворення (shuffle/repeat), Observer для синхронізації програвача з UI).
12. **Система управління навчальним закладом** (сутності: студенти, викладачі, курси, групи; патерни: Composite для структури факультет/група/студент, Visitor для обчислення рейтингів, Factory для типів занять).
13. **Емулятор файлової системи** (сутності: файли, каталоги, дискові томи, права доступу; патерни: Composite для дерева каталогів, Command для операцій copy/move/delete з undo, Proxy для контролю доступу).

14. **Турнірна система спортивної ліги** (сутності: команди, гравці, матчі, сезони; патерни: Template Method для регламентів турнірів, State для стадій (група/плей-оф/фінал), Strategy для формул розрахунку очок).
15. **Симулятор зоопарку або ферми** (сутності: тварини, вольєри, працівники, корми; патерни: Abstract Factory для родин тварин, Observer для моніторингу стану здоров'я, Strategy для режимів годування).
16. **Система кінопрокату та онлайн-кінотеатру** (сутності: фільми, сеанси, квитки, глядачі; патерни: Builder для формування замовлень, Decorator для знижок та бонусів, Chain of Responsibility для перевірок бронювання).
17. **Система розумної безпеки та відеоспостереження** (сутності: камери, датчики руху, сигналізації, оператори; патерни: State для режимів "охорона/тривога/вимкнено", Mediator для координації пристроїв, Observer для сповіщень).
18. **Планувальник подорожей та турагентство** (сутності: тури, маршрути, бронювання, клієнти; патерни: Builder для збирання комплексного туру (переліт+готель+екскурсії), Strategy для розрахунку вартості, Memento для збереження чернеток).
19. **Платформа онлайн-навчання (LMS)** (сутності: курси, уроки, тести, прогрес студента; патерни: State для статусів проходження курсу, Strategy для алгоритмів оцінювання, Observer для сповіщень про дедлайни).
20. **Чат-бот та система обміну повідомленнями** (сутності: користувачі, чати, повідомлення, команди бота; патерни: Command для обробки команд ботом, Chain of Responsibility для конвеєра фільтрів/модерації, Mediator для координації учасників чату).

РЕКОМЕНДОВАНА ЛІТЕРАТУРА ДО КУРСУ

1. Freeman E., Robson E. *Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software*. – 2nd ed. – O'Reilly Media, 2020.
2. Martin R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. – Prentice Hall, 2008.
3. Martin R. C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. – Prentice Hall, 2017.
4. Price M. J. *C# 11 and .NET 7 – Modern Cross-Platform Development Fundamentals*. – Packt Publishing, 2022.
5. Osherove R. *The Art of Unit Testing: with examples in C#*. – 2nd ed. – Manning, 2013.
6. Microsoft. *C# Documentation*. – Електрон. ресурс. – Режим доступу: <https://learn.microsoft.com/en-us/dotnet/csharp/>
7. Refactoring.Guru. *Design Patterns*. – Електрон. ресурс. – Режим доступу: <https://refactoring.guru/design-patterns>

Розділ I. Архітектурні основи ООП та проектування

Практичне заняття 1. Моделювання предметної області та каркас проекту

Аналіз вимог та розробка UML-діаграм класів для системи. Побудова базової архітектури C#-рішення, розподіл по проектах (Domain, App, Tests) і namespace.

- **Самостійна робота 1.** Ініціалізація та життєвий цикл об'єкта. Налаштувати Git-репозиторій, **.gitignore** і **README**. Перекрити життєвий цикл об'єктів через систему конструкторів (основний, копіювальний, з параметрами) та деструктори/Dispose де доречно.

Практичне заняття 2. Інкапсуляція та цілісність даних

Забезпечення консистентності внутрішнього стану системи за допомогою **private**-полів та властивостей з валідацією. Винесення інваріантів на рівень сетерів і конструкторів.

- **Самостійна робота 2.** Агрегація даних. Використати індексатори та перевантаження операторів (+, ==, !=) для агрегації й маніпулювання даними безпосередньо через об'єкти. Переконайтеся, що операції не порушують інваріантів.

Практичне заняття 3. Наслідування та розширення поведінки

Проектування ієрархій сутностей предметної області. Поліморфна поведінка ключових методів через **virtual** та **override**, використання **base** для розширення поведінки батька.

- **Самостійна робота 3.** Перевизначення vs приховування. Дослідити кейси використання **override** (поліморфізм) у порівнянні з приховуванням поведінки **new**. Задokumentувати, чому поліморфна поведінка зазвичай коректніша.

Практичне заняття 4. Контрактне програмування

Розділення системи на слабозв'язані підсистеми через інтерфейси. Винесення спільної поведінки в абстрактні класи. Розмежування, коли доречний інтерфейс, а коли абстрактний клас.

- **Самостійна робота 4.** Проектування інтерфейсів взаємодії. Виділити ізольовані контракти без прив'язки до реалізацій базових класів. Перевірити, що компоненти можна підмінити альтернативною імплементацією.

Розділ II. Структури даних, запити та обробка помилок

Практичне заняття 5. Узагальнені типи (Generics)

Створення універсальних сховищ (наприклад **Repository<T>**) або контейнерів із where-обмеженнями. Уникнення дублювання коду через типобезпечне повторне використання.

- **Самостійна робота 5.** Узагальнені алгоритми та функціональні делегати. Реалізувати делегування методів за допомогою **Action** та **Func**. Написати універсальні алгоритми обробки (**ForEach**, **Map**, **Reduce**) на базі делегатів.

Практичне заняття 6. Колекції .NET та оптимізація зберігання

Обґрунтування використання **List<T>**, **Dictionary<TKey, TValue>**, **HashSet<T>** для різних типів та обсягів даних застосунку. Правильний вибір структури під задачу.

- **Самостійна робота 6.** Оптимізація структур даних. Порівняти швидкодію пошуку та вставки для різних сценаріїв зберігання. Зафіксувати результати вимірювань і висновки щодо вибору.

Практичне заняття 7. Інтегровані запити (LINQ)

Фільтрація, агрегація та ефективна проекція структур даних з використанням декларативних LINQ-запитів. Порівняння **method syntax** та **query syntax**.

- **Самостійна робота 7.** Складні LINQ-запити. Побудувати ланцюжки запитів з **GroupBy**, **Join**, **Aggregate**. Винести часто вживану логіку вибірки в спеціалізовані Extension Methods.

Практичне заняття 8. Система винятків та стратегії обробки

Визначення ієрархії Custom Exceptions для предметної області. Централізоване управління **try/catch**, коректне використання **finally** та **using**.

- **Самостійна робота 8.** Механізми відновлення (Retry Policy). Реалізувати алгоритми відновлення після збоїв та надійне логування подій без впливу на основний цикл програми. Додати експоненційну затримку між спробами.

Розділ III. SOLID та патерни проєктування

Практичне заняття 9. Рефакторинг за SRP та OCP

Розділення відповідальностей (переміщення логіки виконання окремо від логіки відображення). Забезпечення розширюваності системи без модифікації коду бази.

- **Самостійна робота 9.** Зменшення зв'язності (LSP, ISP, DIP). Вирішити проблему заміності базових класів та впровадити інверсію залежностей для зовнішніх сервісів. Перевірити, що клієнти не залежать від конкретних реалізацій.

Практичне заняття 10. Уніфікація створення об'єктів: породжувальні патерни

Ізоляція логіки створення сутностей через Factory Method та патерн Singleton. Мотивація застосування кожного з патернів.

- **Самостійна робота 10.** Застосування фабричних методів у модулях. Реалізувати динамічне створення підтипів і механізмів на основі зовнішніх конфігурацій (JSON або параметри CLI).

Практичне заняття 11. Делегування: поведінкові патерни

Підміна алгоритмів дій під час виконання (Strategy). Реакція системи на зміни стану об'єктів (Observer). Розв'язка "хто викликає" від "що саме виконується".

- **Самостійна робота 11.** Система подій та підписок. Розв'язати компоненти через івент-приховані системи сповіщень (**event**, **EventHandler<T>**). Перевірити, що підписники можуть бути відключені без ризику memory leak.

Практичне заняття 12. Масштабування: структурні патерни

Динамічне розширення та зміна функціоналу компонентів (Decorator). Створення єдиного спільного входу до складних підсистем (Facade).

- **Самостійна робота 12.** Композитне групування. Використати патерн Composite для єдиного керування структурованими системами (дерева, ієрархії, групи сутностей).

Розділ IV. Серіалізація, тестування та життєвий цикл проєкту

Практичне заняття 13. Зберігання стану програми (JSON)

Реалізація збереження та відновлення стану складних об'єктів середовища за допомогою `System.Text.Json`. Робота з опціями серіалізації та циклічними посиланнями.

- **Самостійна робота 13.** DTO і проблеми вкладеності. Розділити доменні моделі та спеціалізовані DTO (Data Transfer Objects) для безпечного збереження даних. Налаштувати маппінг між доменом і DTO.

Практичне заняття 14. Модульне тестування бізнес-логіки

Використання тестових фреймворків (JUnit або xUnit) для автоматизованої перевірки критичних вузлів. Структура AAA (Arrange-Act-Assert), параметризовані тести.

- **Самостійна робота 14.** Ізоляція тестів за допомогою моків (Mock). Впровадити Mock-об'єкти для зовнішніх залежностей (файлова система, генератори потоків/рандому, логери). Перевірити, що тести не залежать від середовища.

Практичне заняття 15. Аналіз коду та підсумковий рефакторинг

Аудит бази коду, усунення архітектурних недоліків (code smells, магічні числа), форматування стилю та неймінгу. Узгодження із правилами Clean Code.

- **Самостійна робота 15.** Документація проєкту. Фінально оформити `README`: архітектура, UML-діаграми, перелік застосованих патернів, інструкція із запуску та прикладів використання. Додати CHANGELOG за потреби.

Практичне заняття 16. Демонстрація та архітектурний захист

Презентація готового та відтестованого рішення. Захист застосованих принципів ООП та обґрунтування обраних патернів: чому саме Strategy, а не switch; чому Factory, а не `new`; де саме SOLID дав виграв у гнучкості.